Aufgabensammlung zur Prüfungsvorbereitung

Name, Vorname: _____

Allgemeine Hinweise:

1) Erlaubte Hilfsmittel: 2 Seiten A4 mit Notizen
2) Nicht erlaubt: Elektronische Geräte (Handy, Taschenrechner, Kopfhörer), Kommunikation mit anderen Personen
3) Schreibe die Antworten direkt auf das Aufgabenblatt. Schreibe Deinen Namen auch auf alle Zusatzblätter.
4) Bewertet werden die Korrektheit der Resultate sowie die Herleitung / Begründung.
5) Schreibe nicht mit roter Farbe.

Viel Erfolg!

Die folgenden Aufgaben wurden so oder in ähnlicher Form einmal in Prüfungen gestellt. Um Dir einen realistischen Eindruck zu geben, solltest Du diese Aufgaben unter selbstkontrollierten prüfungsgleichen Bedingungen bearbeiten – und bis dahin nicht anschauen.

Verwende nur Hilfsmittel, die Dir auch an der tatsächlichen Prüfung zur Verfügung stehen.

Wende Dich an Deinen Dozenten für ein inhaltliches Feedback zu Deinen Lösungen.

Aufgabe 1: Test einer Bedingung **(4 Punkte)**

Nehmen wir an, eine Methode erwarte als Parameter ein `int`-Array `x` mit der Eigenschaft, dass jedes Element ein anderes Vorzeichen habe als das unmittelbar vorangegangene. Wenn also gilt `x[0]` < 0, dann muss gelten `x[1]` ≥ 0 und `x[2]` < 0 usw. oder umgekehrt, wenn `x[0]` ≥ 0, dann gilt `x[1]` < 0 und `x[2]` ≥ 0 usw.

Schreibe ein Java-Programm, das diese Bedingung mit sequenzieller Suche überprüft.

Überlege dazu zunächst, wonach genau das Programm suchen soll!

```
boolean isOK(int[] x) {




}
```

Aufgabe 2: Asymptotische Komplexität 1 **(3 Punkte)**

Du hast die *O*-Notation als Werkzeug kennengelernt, mit dem Funktionen nach ihrem Wachstumsverhalten für annähernd ins Unendliche wachsende Parameter klassifiziert werden können.

Wende dieses Werkzeug an und gib zu den folgenden Funktionen jeweils eine möglichst einfache aber nicht zu gross bemessene Komplexitätsklasse mit der *O*-Notation an (es werden hier *keine Begründungen* erwartet):

a) [*2 Punkte*]

$$f_1(n) = \sum_{i=0}^{n} i$$

$$f_2(n) = \frac{9\,n}{3\,\sqrt{n}} + 27$$

$$f_3(n) = 4\,log_2(n) + \frac{16}{n}$$

$$f_4(n) = n + 4\,log_{10}(n)$$

$$f_5(n) = n^3 + n^2 + n + 1$$

$$f_6(n) = 23(log_{10}(n) + 12)$$

$$f_7(n) = 2^n + 3^n + 4^n$$

$$f_8(n) = 65 \cdot n \cdot log_2(n) + 1$$

b) [*1 Punkt*] Ordne die Funktionen f_1 bis f_8 aus a) aufsteigend nach ihrem asymptotischen Wachstum. Gruppiere gleich schnell wachsende Funktionen dabei klar erkennbar.

Aufgabe 3: Asymptotische Laufzeitkomplexität 2 **(3 Punkte)**

Gib für folgende Code-Fragmente die *asymptotische Laufzeitkomplexität* mithilfe der *O*-Notation an. Die Laufzeit der nicht dargestellten Operationen innerhalb der innersten Schleifen sei jeweils in *O*(1).

a)

```
for (int i = 0; i*i < n; ++i) {
    for (int j = n; j > 0; --j) {
       // ...
    }
}
```

b)

```
for (int i = 0; i < n / 5; i++) {
    for (int j = n; j > 0; j/=2) {
      // ...
    }
}
```

c)

```
for (int i = 0; i < 1000; ++i) {
  for (int j = 1; j <= 42; ++j) {
    for (int k = 0; k < 2*n; ++k) {
      // ...
    }
  }
}
```

Aufgabe 4: Sortieren **(9 Punkte)**

Du hast die elementaren Sortierverfahren *Selection-Sort* und *Insertion-Sort* kennengelernt. Eines dieser Verfahren passt zu folgender Beschreibung, wenn auch in einem gewissen Sinne «andersherum» als Du es kennengelernt hast:

Sortiert wird in einem Array *a*. Das Programm besteht aus zwei Schleifen ineinander. Die *äussere* der beiden Schleifen arbeitet mit einer Variablen *i*. Vor jedem neuen Durchlauf dieser Schleife gilt jeweils:

$$(\forall\, j : i < j < a.length : a[\, j-1\,] \leq a[\, j\,]).$$

Weitere Aussagen lassen sich nicht machen, ausser natürlich, dass die Elemente durch den Algorithmus im Array nur miteinander vertauscht werden und keine entfernt oder hinzugefügt werden. Beispielsweise könntest Du also folgendes sehen:

13	3	23	15	29	7	9	17	25	30
0					i				

a) [*1 Punkt*] Welcher der beiden oben genannten Algorithmen steckt hier als *Strategie* dahinter?

b) [*1 Punkt*] Wie sieht das obige Beispiel-Array nach dem nächsten Durchlauf der äusseren Schleife aus?

c) [*1 Punkt*] Welchen Wert hat die Variable *i*, wenn die Sortierung vollständig abgeschlossen ist?

d) [*1 Punkt*] Mit welchem Wert muss die Variable *i* ganz am Anfang starten?

e) [*5 Punkte*] Programmiere das hier dargestellte Sortierverfahren auf der Basis der in den vorherigen Teilaufgaben gewonnenen Erkenntnissen.

```
void sort(int[] a) {



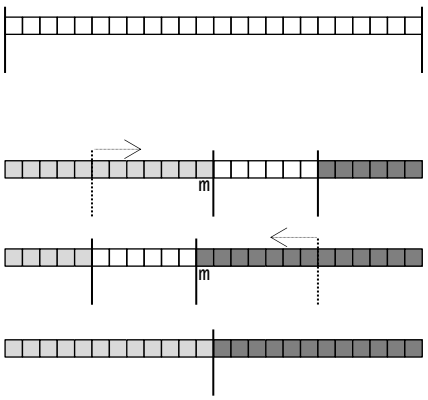

}
```

Aufgabe 5: Fehlerhaftes Programm **(9 Punkte)**

Das folgende Programm funktioniert nicht in allen Fällen fehlerfrei. Korrigiere das, indem Du an *genau einer Stelle* etwas änderst.

a) [*4 Punkte*] Ergänze zur Fehleranalyse die schematischen Skizzen, so dass sie die im vorliegenden Programm an den jeweiligen Stellen getroffenen Annahmen zeigen.

```
int myProg (ArrayList<Thing> list) {
   int left = 0;
   int right = list.size();
   while (left <= right) {
      int m = left + (right - left) / 2;
      if (list.get(m).color.equals("white")) {
         left = m + 1;
      } else {
         right = m - 1;
      }
   }
   return left;
}
```



b) [*1 Punkt*] Erläutere in einem Satz, woran Du bei den gemäss a) ergänzten Skizzen erkennst, dass das Programm fehlerhaft ist.

c) [*2 Punkte*] Markiere *genau einen Ort* im Programm und gib an, was dort stehen sollte, damit das Programm in allen Fällen korrekt funktioniert.

d) [*2 Punkte*] Beschreibe in Worten oder mit je einem logischen Ausdruck die vom Programm erwartete Vorbedingung und das vom Programm berechnete Resultat.

Aufgabe 6: Programme mit Collections **(4 Punkte)**

Welche der folgenden Codestücke kompilieren? Wenn nicht, weshalb? Gib für jene Codestücke, die kompilieren, die Ausgabe an. Falls eine Exception geworfen wird, nenne den Grund dafür.

a)
```
ArrayList<String> list = new ArrayList<>();
list.add("one");
list.add("two");
list.remove("three");
IO.println(list);
```

b)
```
HashSet<String> set = new HashSet<>();
set.add("one");
set.add("two");
set.add("two");
IO.println(set);
```

c)
```
HashMap<String, Integer> map = new HashMap<>();
map.put("foo", 1);
map.put("bar", 2);
map.put("bar", 1);
IO.println(map.size());
IO.println(map.keySet().size());
IO.println(map.values().size());
```

d)
```
Map<Integer, String> map = Map.of(1, "one", 2, "two", 3, "three");
IO.println(map.get(2));
IO.println(map.get(4));
```

Aufgabe 7: Auswahl von Collection-Klassen **(4 Punkte)**

Die Statistikdatei eines Webshops enthält aus vielen Bestellungen die bestellten Artikel. In jeder Zeile der Text-Datei steht eine *Artikelnummer* sowie die *Anzahl* wie oft dieser Artikel in jeweils einer Bestellung enthalten war. Ein bereits bestehendes Programm liest die Datei zeilenweise, extrahiert die erwähnten Informationen und ruft eine Methode `void register(String article, int number)` damit auf.

Nehmen wir an Du sollst diese Methode `register` als Teil einer eigenen Klasse implementieren, deren Objekte jeweils eine der folgenden Aufgaben lösen sollen:

a) Das Objekt mit der Methode `register` soll auf Anfrage hin die *Anzahl verschiedener Artikel* überhaupt, die je bestellt wurden ermitteln.

   Welche Collection-Klasse wird Dein Objekt dafür verwenden?

   Begründe Deine Antwort!

b) Das Objekt mit der Methode `register` soll auf Anfrage hin für jeden bestellten Artikel Auskunft geben können, *wie oft dieser Artikel insgesamt bestellt* wurde.

   Welche Collection-Klasse wird Dein Objekt dafür verwenden?

   Begründe Deine Antwort!

Aufgabe 8: Objekte und Referenzen **(10 Punkte)**

Gegeben sei die folgende Klasse und Client Code:

```
public class Movie {
   private final String title;
   private Movie sequel;

   public Movie(String title) {
      this.title = title;
   }

   public String getTitle() { return title; }

   public Movie getSequel() { return sequel; }

   public void setSequel(Movie sequel) {
      this.sequel = sequel;
   }

   public String toString() {        // diese Methode wird von IO.println aufgerufen
      return "Movie \"" + title + "\"";
   }
}

void main(String[] args) {
   Movie ironMan = new Movie("Iron Man");
   Movie ironMan2 = new Movie("Iron Man 2");
   Movie thor = new Movie("Thor");
   Movie cap = new Movie("Captain America: The First Avenger");
   Movie avengers = new Movie("The Avengers");
   Movie inception = new Movie("Inception");
   Movie primer = new Movie("Primer");

   ironMan.setSequel(ironMan2);
   ironMan2.setSequel(avengers);
   thor.setSequel(avengers);
   cap.setSequel(avengers);
   primer.setSequel(primer);

   IO.println(/* HIER */);
}
```

Gib für jeden der Ausdrücke auf der nächsten Seite an, welche Ausgabe das Programm erzeugen würde, wenn man den Ausdruck in der `main`-Methode an der Stelle `/* HIER */` einfügen würde. Falls bei der Ausführung ein Fehler auftritt, schreibe «Fehler» hin.

a) `ironMan`

b) `ironMan.getSequel().getSequel()`

c) `cap.getSequel().getSequel()`

d) `thor.getSequel() == cap.getSequel()`

e) `thor.getSequel().equals(cap.getSequel())`

f) `ironMan.getSequel().getSequel().getSequel().getTitle() == null`

g) `primer.getSequel().getTitle() == "Movie \"Primer\""`

h) `ironMan2.getSequel().getSequel() == avengers.getSequel()`

i) `inception.getSequel() == avengers.getSequel()`

j) `inception.getSequel().getTitle().equals(avengers.getSequel().getTitle())`

Aufgabe 9: Klasseninvarianten & Datenkapselung **(11 Punkte)**

In dieser Aufgabe programmierst Du eine Klasse `Amount`, welche einen Geldbetrag in der Zauberwelt repräsentiert. Wie in der Schweiz ein Geldbetrag aus Franken und Rappen besteht, besteht ein Zaubergeldbetrag aus Galleons, Sickles und Knuts. 29 Knuts sind ein Sickle wert und 17 Sickles entsprechen 1 Galleon.

Für die Klasse sollen folgende Invarianten gelten:

- Geldbeträge sind unveränderlich und nicht negativ.
- Der Getter für Sickles gibt immer Werte zwischen 0 und 16 (inklusive) zurück.
- Der Getter für Knuts gibt immer Werte zwischen 0 und 28 (inklusive) zurück.

a) [*3 Punkte*] Formuliere eine möglichst wenig restriktive Precondition für den Konstruktor mit folgender Signatur: `public Amount(int galleons, int sickles, int knuts)`. Das resultierende `Amount`-Objekt soll die Summe der gegebenen Teilbeträge repräsentieren.

b) [*8 Punkte*] Vervollständige das Skelett der `Amount`-Klasse auf der folgenden Seite, sodass die Invarianten eingehalten sind und von ausserhalb der Klasse nicht verletzt werden können. Wenn die Konstruktor-Precondition verletzt wird, soll eine `IllegalArgumentException` geworfen werden.

```
public class Amount {

   public Amount(int galleons, int sickles, int knuts) {

   }
   public int getGalleons() {

   }
   public int getSickles() {

   }
   public int getKnuts() {

   }
   public Amount plus(Amount other) {

   }
}
```